Documento de cobertura del testing

Sistema Ch.Store — Frontend React y backend

Asignatura: DSY1104 — Desarrollo Full Stack II
Evaluación: Parcial 2 (30%)
Versión: 2.1
Fecha: 28 de octubre de 2026
Autores: Javiera Rojas, Mhyjal Rozas

---

1. Resumen ejecutivo

Frontend (Jasmine y Karma)

Indicador	Resultado
Pruebas unitarias escritas	**188**
Pruebas aprobadas	**188 (100%)**
Pruebas fallidas	0
Cobertura de sentencias	94,26% (937 de 994)
Cobertura de ramas	88,05% (553 de 628)
Cobertura de funciones	93,08% (350 de 376)
Cobertura de líneas	**96,08% (810 de 843)**
Tiempo de ejecución	cerca de 3 segundos
Entorno	Karma 6 con Jasmine, Chrome Headless

Backend (`node:test`)

Indicador	Resultado
Pruebas escritas y aprobadas	**33 de 33 (100%)**
Cobertura de líneas	98,37%
Cobertura de ramas	92,13%
Cobertura de funciones	98,57%
Tiempo de ejecución	cerca de 11 segundos (el cifrado de contraseñas es deliberadamente lento)
Entorno	Node 24, base SQLite en memoria

Los resultados se obtuvieron el 28 de octubre de 2026 con `npm test` en `frontend-react/` y con `npm test` en `backend/` (la cobertura del backend con `--experimental-test-coverage`). El reporte navegable del frontend se genera en `frontend-react/coverage/` (HTML y `lcov.info`).

Cambios respecto de la versión anterior de este documento: las pruebas del frontend pasaron de 138 a 188 y la cobertura de líneas de 95,25% a 96,08%, porque se agregaron las pruebas del modo API y de la autenticación en pantalla. El backend es nuevo y trae sus propias 33 pruebas.

2. Configuración del entorno de pruebas

2.1 Frontend

Elemento	Detalle
Framework de pruebas	Jasmine (`jasmine-core`, `karma-jasmine`)
Ejecutor	Karma (`karma.conf.cjs`), con `ChromeHeadless --no-sandbox`
Compilación	`karma-webpack` con `babel-loader` y los presets `@babel/preset-env` y `@babel/preset-react` (JSX automático)
Cobertura	`babel-plugin-istanbul` instrumenta `src/` (excluye `src/tests/`) y `karma-coverage` genera los reportes `html`, `lcov` y resumen de texto
Reporte en consola	`karma-spec-reporter`
Punto de entrada	`src/tests/index.js` carga automáticamente todos los `*.spec.js` y `*.spec.jsx`
Utilidades	`src/tests/helpers.jsx`: `montar` (renderiza en el DOM real, con enrutador y contextos opcionales), `clic`, `escribir`, `elegir` y `enviar` para simular al usuario

Las pruebas corren en un navegador real, por lo que se verifica el DOM verdadero. Karma busca Chrome en la ruta de macOS; en otro sistema se indica con la variable de entorno `CHROME_BIN`.

```
cd frontend-react
npm install
npm test            # una ejecución, con reporte de cobertura
npm run test:watch  # modo continuo
```

2.2 Backend

Elemento	Detalle
Framework de pruebas	`node:test` y `node:assert`, incluidos en Node (sin dependencias extra)
Servidor	La API real de Express se levanta en un puerto libre dentro de la prueba y se consulta con `fetch`
Base de datos	SQLite en memoria (`:memory:`), reiniciada con los datos iniciales antes de cada prueba
Cobertura	`node --test --experimental-test-coverage`
Archivo	`backend/test/api.test.js`

```
cd backend
npm install
npm test
```

Desde la raíz del repositorio, `npm test` ejecuta primero las pruebas del backend y después las del frontend.

3. Técnicas de aislamiento y mocks

Técnica	Dónde se usa	Qué resuelve
Espía de función (`jasmine.createSpy`)	Botón, selector de región y avisos de sesión vencida	Verifica que el componente invoque el `onClick`, `onChange` o el aviso recibido, y cuántas veces
Servidor simulado: `spyOn(window, 'fetch')` con `callFake`	`db-api.spec.js` y `auth-api.spec.jsx`	Simula las respuestas de la API (login correcto, 401, 409, caída de conexión) sin levantar el backend; las llamadas quedan registradas y se revisan sus rutas, métodos y cuerpos
Espía sobre método existente (`spyOn(window, 'print')`)	Boleta de compra	Evita abrir el diálogo de impresión y comprueba la llamada
Espía sobre `console.error`	Datos corruptos y errores de guardado esperados	Comprueba que el error se registre sin ensuciar la consola
Dependencia inyectada	`generarCodigoEnvio(aleatorio)` y `crearLimitador({ ahora })`	Un generador aleatorio falso y un reloj controlado hacen deterministas el código de envío y el bloqueo por intentos
Fuente de datos reiniciable	`db.reiniciar()` en el frontend y `reiniciar(db)` en el backend	Cada prueba parte con los datos iniciales; `localStorage` se limpia entre pruebas
Datos sembrados	Carrito y sesión escritos en `localStorage` antes de montar	Crea el contexto exacto que necesita cada caso
Enrutador en memoria (`MemoryRouter`)	Todas las pruebas de vistas	Navegar entre rutas sin depender de la URL del navegador
Rutas de destino simuladas	Checkout y control de acceso	Verifica adónde navega el componente sin montar la pantalla de destino

4. Pruebas por archivo

4.1 Frontend (`frontend-react/src/tests/`)

Archivo	Pruebas	Qué verifica
`validaciones.spec.js`	12	RUN (módulo 11, dígito K y 0, formato y largo), correo y dominios, requerido, rango, enteros, encadenado de reglas y `validarCampos`
`carrito.spec.js`	13	Reglas R1–R5 del carrito, precio con oferta, totales, cupones, unidades y código de envío con aleatorio simulado
`db.spec.js`	14	CRUD completo sobre `localStorage`, identificadores duplicados, registros inexistentes, suscripción a cambios, datos corruptos, ofertas, críticos, categorías, registro de usuarios, historial y descuento de stock
`db-api.spec.js`	35	Modo API con servidor simulado: conexión y respaldo local, login por perfil y datos que carga cada uno, token en las peticiones, sesión vencida, restauración de la sesión, escrituras (crear, actualizar, eliminar, orden), reversión cuando el servidor rechaza un guardado, reinicio, registro de usuarios y la utilidad `alResolver`
`componentes.spec.jsx`	14	Renderizado de listas, renderizado condicional (error, stock, oferta, descuento), props (etiqueta, `onClick`, enlace, error), estado de formularios y eventos

Archivo	Pruebas	Qué verifica
`paginas.spec.jsx`	30	Productos y filtros, ofertas, detalle, carrito, cupones, checkout, pago aprobado y rechazado, boleta, login, registro, contacto y control de acceso por rol
`auth-api.spec.jsx`	15	Pantallas con el backend simulado: login que entra al panel o a la tienda según el perfil, error de contraseña, cierre de sesión, sesión vencida en pantalla, registro con contraseña y con correo repetido, dashboard del vendedor sin tarjeta de usuarios y formulario de usuario con contraseña (obligatoria al crear, opcional al editar)
`admin.spec.js`	11	Permisos por rol, productos críticos, indicadores, ventas por producto, órdenes por estado y reglas de validación del panel y del checkout
`admin-paginas.spec.jsx`	23	Dashboard, menú por rol, mantenedores de productos, categorías y usuarios, formularios, historial de compras, órdenes, boleta, reportes y perfil
`navegacion.spec.jsx`	21	13 rutas públicas (una prueba por ruta) y flujos: ruta desconocida, carrito en el encabezado, buscador, redirección sin sesión, inicio y cierre de sesión

Total: 12 + 13 + 14 + 35 + 14 + 30 + 15 + 11 + 23 + 21 = **188** pruebas.

4.2 Backend (`backend/test/api.test.js`)

Grupo	Pruebas	Qué verifica
Datos y CRUD	12	Datos iniciales, orden de las órdenes, crear, leer, actualizar y eliminar productos y categorías, código y correo repetidos, categoría inexistente o con productos, descuento de stock en órdenes aprobadas (y no en rechazadas), stock que nunca baja de cero, rutas y JSON inválidos, y reinicio
Inicio y cierre de sesión	9	Login con token y sin contraseña en la respuesta, mismo mensaje para usuario inexistente y contraseña mala, validación del tipo de los datos, bloqueo tras 5 fallos y liberación con el tiempo, reinicio del contador, contraseñas y tokens que no quedan en claro, `GET /auth/yo`, cierre de sesión y token vencido
Registro	2	El registro público solo crea clientes y permite luego iniciar sesión; validación de la contraseña y rechazo de RUN y correo repetidos
Permisos por perfil	10	Sin sesión (401) y con una sesión sin permiso (403) para cada operación protegida, el catálogo público, lo que puede hacer el vendedor y el cliente, compra sin sesión con estados permitidos, administrador que crea usuarios con contraseña y la restablece, edición del perfil propio sin poder cambiar el perfil de acceso, el correo ni la contraseña, cuenta propia que no se puede eliminar y sesiones cerradas al eliminar un usuario

Total: 12 + 9 + 2 + 10 = **33** pruebas.

5. Correspondencia con los tipos de prueba propuestos por el docente

Tipo	Tarea propuesta	Prueba que la cubre
Renderizado correcto	Componentes de lista renderizan todos los elementos	`ProductList renderiza todos los elementos del conjunto de datos`
Renderizado condicional	Mensaje de error solo si hay error	`MensajeError solo se muestra cuando hay un error presente`
Propiedades	Botón recibe etiqueta y `onClick`	`Boton recibe la etiqueta y ejecuta la función onClick`
Estado	El estado del formulario cambia al escribir	`el estado del formulario cambia cuando el usuario introduce texto`

Tipo	Tarea propuesta	Prueba que la cubre
Eventos	Un clic cambia el estado o ejecuta una función	`un clic cambia el estado del componente` y los espías de `onClick`
Mocks	Reemplazar una dependencia por una controlada	`si el servidor rechaza un guardado vuelve a leer la colección` (servidor simulado con `fetch`)

6. Cobertura por archivo

Porcentajes de líneas, ramas y funciones del último reporte del frontend.

Lógica y datos

Archivo	Líneas	Ramas	Funciones
`utils/validaciones.js`	97%	86%	100%
`utils/carrito.js`	96%	82%	100%
`utils/pago.js`	100%	100%	100%
`utils/admin.js`	100%	94%	100%
`utils/regiones.js`, `utils/formato.js`, `utils/asincrono.js`	100%	100%	100%
`data/db.js`	98%	95%	97%
`data/blogs.js`, `data/productosBase.js`	100%	100%	100%
`context/CarritoContext.jsx`	97%	88%	91%
`context/SesionContext.jsx`	97%	100%	100%
`hooks/useColeccion.js`	100%	100%	100%

Componentes

Archivo	Líneas	Ramas	Funciones
`Alerta`, `Boton`, `CampoFormulario`, `Footer`, `LayoutAdmin`, `LayoutTienda`, `MensajeError`, `Precio`, `ProductList`, `ResumenTotales`, `RutaAdmin`, `SelectorRegionComuna`, `TablaItems`	100%	100%	100%
`ProductCard.jsx`	100%	90%	100%
`Buscador.jsx`	100%	50%	100%
`TarjetaIndicador.jsx`	100%	80%	100%
`Navbar.jsx`	80%	100%	75%
`FilaCarrito.jsx`, `ConfirmarEliminar.jsx`	83%	100%	75%

Páginas de la tienda

Archivo	Líneas	Ramas	Funciones
`Home`, `Blogs`, `Categorias`, `Contacto`, `Nosotros`, `Ofertas`	100%	100%	100%
`Login.jsx`	100%	100%	100%
`Productos.jsx`	95%	93%	80%

Archivo	Líneas	Ramas	Funciones
`Registro.jsx`	95%	74%	91%
`Checkout.jsx`	94%	92%	88%
`Carrito.jsx`	91%	100%	83%
`DetalleBlog.jsx`	88%	88%	80%
`ResultadoCompra.jsx`	88%	82%	100%
`DetalleProducto.jsx`	79%	93%	56%

Páginas del panel

Archivo	Líneas	Ramas	Funciones
`Dashboard.jsx`	100%	100%	100%
`HistorialCompras.jsx`	100%	50%	100%
`Ordenes.jsx`	100%	75%	100%
`CategoriaForm.jsx`	100%	56%	100%
`ProductoForm.jsx`	100%	83%	86%
`UsuarioForm.jsx`	94%	88%	83%
`Perfil.jsx`	94%	60%	82%
`Usuarios.jsx`	93%	100%	88%
`Productos.jsx`	93%	94%	83%
`Categorias.jsx`	91%	50%	86%
`Boleta.jsx`	90%	62%	75%
`Reportes.jsx`	89%	100%	67%

Backend

Archivo	Líneas	Ramas	Funciones
`app.js` (rutas, permisos y autenticación)	99,03%	93,75%	95,83%
`repositorio.js` (CRUD y órdenes)	95,88%	88,68%	100%
`db.js` (esquema y datos iniciales)	100%	81,25%	100%
`seguridad.js` (cifrado, tokens y límite de intentos)	100%	100%	100%
`sesiones.js`	100%	100%	100%

El archivo `server.js` es solo el arranque del servidor y no se mide.

7. Análisis de resultados

Fortalezas

- Toda la lógica de negocio (validaciones, carrito, pago, indicadores) y la capa de datos superan el 95% de líneas. Se probó porque son funciones puras, sin DOM ni almacenamiento.

- La seguridad del backend está cubierta casi por completo: cifrado de contraseñas, tokens, vencimiento, bloqueo por intentos y los permisos de cada perfil en cada ruta. `seguridad.js` y `sesiones.js` están al 100%.
- La capa de datos del frontend se prueba en sus dos modos: `localStorage` y API, este último con un servidor simulado, de modo que las pruebas no dependen de tener el backend encendido.
- Los flujos críticos están cubiertos de punta a punta: la compra (carrito, cupón, checkout, pago aprobado y rechazado, boleta), el inicio de sesión y el control de acceso por perfil.
- Cada requisito funcional tiene al menos una prueba asociada (columna "Pruebas" del ERS V2).

Brechas conocidas y su causa

Brecha	Causa	Riesgo
Ramas bajas en formularios del panel (`CategoriaForm`, `Perfil`, `Boleta`, `Categorias`)	Combinaciones de edición contra alta y estados poco frecuentes no tienen prueba propia	Bajo: las validaciones están cubiertas en `utils/`
Ramas bajas en `Registro.jsx` (74%)	Faltan combinaciones de errores simultáneos entre campos	Bajo: cada regla se prueba por separado en `validaciones.spec.js`
Funciones sin ejecutar en `DetalleProducto` y `Navbar`	Manejadores de la galería de miniaturas y del cierre del aviso, y el menú de usuario según perfil	Bajo: son acciones de presentación
`main.jsx` y `server.js` no se miden	Son solo los puntos de arranque; las pruebas montan los componentes y la aplicación de Express directamente	Bajo: se verificaron a mano arrancando todo con `npm run dev`
Líneas sin ejecutar en `repositorio.js` y `app.js`	Caminos de error poco frecuentes: registro sin campo identificador, campos obligatorios ausentes y la respuesta 500 ante un fallo inesperado	Bajo

Verificación manual complementaria. Además de las pruebas automáticas, el 28 de octubre de 2026 se recorrió la aplicación en el navegador contra el backend real: compra con pago rechazado, reintento y pago aprobado, con comprobación en la base de que la orden se guardó y el stock bajó; alta, edición y eliminación de un producto, alta de una categoría y de un usuario con contraseña desde el panel; ingreso como vendedor con el menú reducido y respuestas 403 del servidor a las operaciones reservadas; una sesión de administrador falsificada en el navegador, que la aplicación descartó; y el bloqueo (429) del login a partir del sexto intento fallido seguido.

Limitaciones

- Las pruebas son unitarias y de integración de componentes: no hay pruebas automáticas de extremo a extremo ni de accesibilidad; esos recorridos se hicieron a mano.
- El resultado del pago se simula, por lo que no se prueba ninguna pasarela real.
- El backend se prueba contra una base en memoria; no se midió su comportamiento bajo carga.

8. Mejoras propuestas

1. Probar la galería de miniaturas y el cierre de avisos en el detalle de producto.
2. Completar las ramas de edición contra alta en los formularios del panel.
3. Automatizar los recorridos de extremo a extremo (frontend con backend) con un navegador controlado por código, en lugar de hacerlos a mano.
4. Configurar un umbral de cobertura en `karma.conf.cjs` para que la ejecución falle si baja del 90% de líneas.
5. Agregar pruebas de carga y de seguridad más profundas al backend (por ejemplo, intentos de inyección y límites de tamaño de las peticiones).